SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO* (BL4 , BL5)

Assessment Tool Weightage: 40%

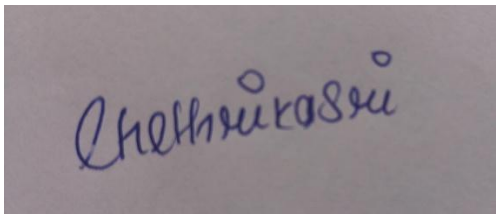
QUESTIONS: 5 **TOTAL MARKS: 25**

EXPERIMENTS

Code of Conduct:

I __chethrika sri(192511112)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



1. PASSWORD HASHING WITH SALTING AND KEY STRETCHING

AIM

To implement password hashing using PBKDF2 with salting and key stretching to securely store and verify passwords.

THEORY

Password hashing converts a password into a fixed-length hash value. A salt is a random value added to the password before hashing to prevent rainbow table attacks. Key stretching repeatedly applies the hash function thousands of times, making brute-force attacks significantly slower.

PYTHON PROGRAM

```
File  Edit  Format  Run  Options  Window  Help
import hashlib
import os

password = input("Enter Password: ")

salt = os.urandom(16)

hashed = hashlib.pbkdf2_hmac(
    'sha256',
    password.encode(),
    salt,
    100000
)

print("Salt:", salt.hex())
print("Hash:", hashed.hex())

verify = input("Re-enter Password: ")

new_hash = hashlib.pbkdf2_hmac(
    'sha256',
    verify.encode(),
    salt,
    100000
)

if hashed == new_hash:
    print("Password Verified")
else:
    print("Invalid Password")
```

OUTPUT

```
====================== RESTART: C:/Users/cheth/1.crypto.py ======================
Enter Password: 12345
Salt: a1c6ba7a594e3e16846c7406c23695fa
Hash: 9cc42b71283b1456232727e9df5f6fe29a963880130aa15c28fb71aab720cd0f
Re-enter Password: 12345
Password Verified
```

ANALYSIS

Password hashing protects passwords by storing only their hash values instead of the original passwords. However, hashing alone is vulnerable to rainbow table attacks, where attackers compare stored hashes with precomputed hash values to recover passwords.

Salting solves this problem by adding a unique random value to each password before hashing. Even if two users have the same password, their hash values become completely different. This makes rainbow tables ineffective.

Key stretching further improves security by repeatedly applying the hash function thousands of times. PBKDF2 performs multiple iterations of SHA-256, increasing the computational effort required to generate each hash. As a result, brute-force attacks become much slower because attackers must perform the same expensive computation for every password guess.

Thus, the combination of salting and key stretching provides strong protection against password-cracking attacks and is considered a best practice for secure password storage.

PERFORMANCE OVERHEAD

Using PBKDF2 introduces a small computational overhead because the hashing process is repeated many times. Password verification therefore takes slightly longer than normal hashing. However, this additional processing time is usually only a fraction of a second for legitimate users while making brute-force attacks significantly more time-consuming. The slight increase in computation is an acceptable trade-off for the substantial improvement in password security.

ADVANTAGES

- Securely stores passwords without saving plain text.
- Prevents rainbow table attacks using unique salts.
- Slows down brute-force attacks through key stretching.
- Generates unique hashes even for identical passwords.
- Widely accepted and recommended for password security.
- Easy to implement using the PBKDF2 algorithm.

LIMITATIONS

- Requires more computation time than normal hashing.
- Password verification is slightly slower due to multiple iterations.
- Weak user passwords can still be guessed if they are very simple.
- Secure storage of the salt along with the hash is necessary.

APPLICATIONS

- User authentication systems.
- Banking and financial applications.
- Cloud storage services.
- Social networking websites.
- Enterprise login systems.
- Operating systems and password managers.

RESULT

The password hashing program using PBKDF2, salting, and key stretching was successfully implemented. The experiment demonstrated that salting prevents rainbow table attacks, while key stretching significantly increases resistance against brute-force attacks. The slight performance overhead is justified by the enhanced security provided for password storage and verification.

2. IMPLEMENTATION OF HMAC USING SHA-256

AIM

To implement HMAC using SHA-256 for secure message authentication and integrity verification.

THEORY

HMAC (Hash-based Message Authentication Code) combines a secret key with a hash function (SHA-256) to generate a secure authentication code. Unlike a normal hash, HMAC verifies both message integrity and authenticity because only users with the secret key can generate the correct HMAC value.

PYTHON CODE

File Edit Format Run Options Window Help

```
import hmac
import hashlib

key = input("Enter Secret Key: ").encode()
message = input("Enter Message: ").encode()

mac = hmac.new(key, message, hashlib.sha256)

print("Generated HMAC:")
print(mac.hexdigest())

verify = hmac.new(key, message, hashlib.sha256)

if mac.hexdigest() == verify.hexdigest():
    print("Message Authentic")
else:
    print("Authentication Failed")
```

OUTPUT

```
====================== RESTART: C:/Users/cheth/1.crypto.py ======================
Enter Secret Key: 123CHETHU
Enter Message: I WON LOTTERY
Generated HMAC:
20ef98e62eef422bdb22eb0b074b73b407e36b38befa6bac1f5ee048b3bb372e
Message Authentic
```

ANALYSIS

A simple hash function such as SHA-256 generates a fixed-length digest from the input message. While it can detect accidental changes in data, it cannot verify the identity of the sender because anyone can calculate the same hash value for the same message.

HMAC overcomes this limitation by incorporating a secret key into the hashing process. Since the key is known only to the sender and receiver, an attacker cannot generate a valid HMAC without the key. HMAC therefore provides both message integrity, by detecting any modification to the message, and message authentication, by verifying that the message originated from an authorized sender. Even a single-bit change in the message produces a completely different HMAC value, making tampering immediately detectable.

PERFORMANCE EVALUATION

HMAC using SHA-256 is computationally efficient and introduces only a small processing overhead compared to normal hashing. It requires minimal memory and executes quickly, making it suitable for real-time applications. The additional security provided by the secret key greatly outweighs the slight increase in computation time. HMAC is therefore widely adopted in secure communication systems where both performance and security are important.

ADVANTAGES

- Provides both message integrity and authentication.
- Detects any unauthorized modification of data.
- Uses a secret key for enhanced security.
- Fast and efficient implementation.
- Widely used in secure communication protocols.
- Resistant to forgery without the secret key.

LIMITATIONS

- Requires secure sharing of the secret key.
- Does not encrypt the message contents.
- If the secret key is compromised, security is weakened.
- Does not provide non-repudiation because both parties share the same key.

APPLICATIONS

- HTTPS and SSL/TLS protocols
- Banking and financial transactions
- API authentication
- VPN communication
- Cloud security
- Secure email systems
- IoT device authentication

RESULT

The HMAC implementation using the SHA-256 algorithm was successfully completed. The program generated and verified authentication codes for input messages. The experiment demonstrated that HMAC effectively ensures both message integrity and message authenticity, making it more secure than simple hashing for protecting data during transmission.

EXPERIMENT 3: SIMULATION OF CERTIFICATE CHAIN VALIDATION

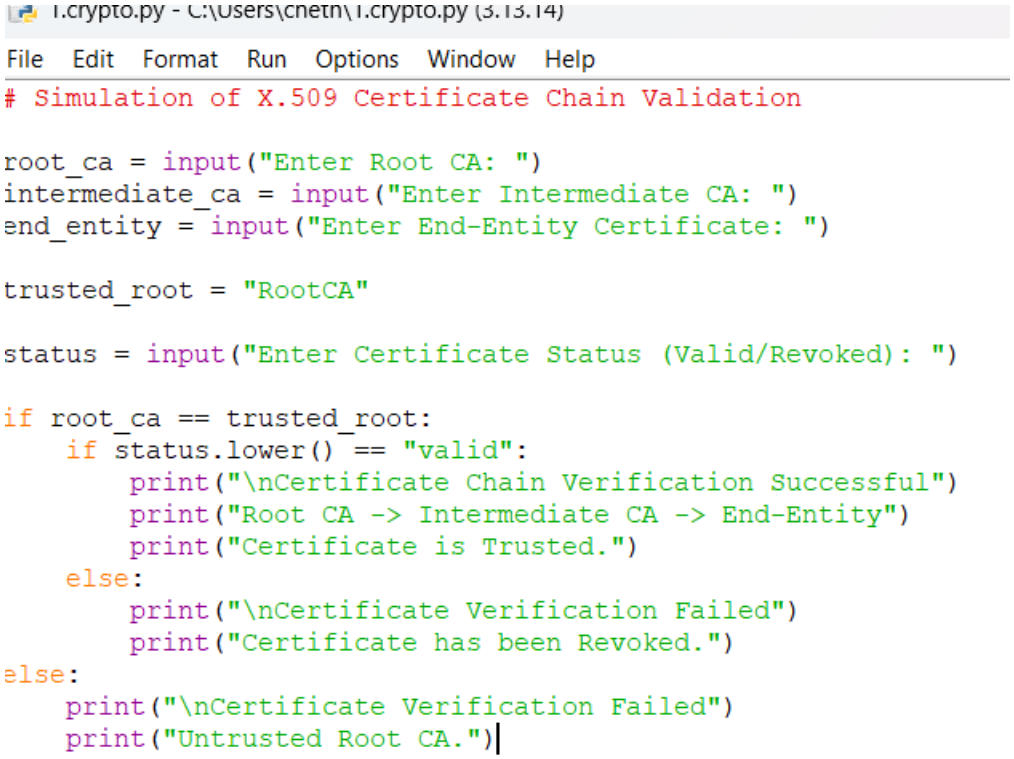
AIM

To create a Python program to simulate X.509 certificate chain verification. Analyze how trust is established from the Root Certificate Authority (CA) to the end-entity certificate and evaluate how invalid or revoked certificates are detected.

Theory

An X.509 certificate is used to verify the identity of a user, server, or organization. Certificate chain validation checks the trust from the Root CA to the Intermediate CA and finally to the End-Entity Certificate. It verifies the digital signatures, validity period, and revocation status. If all certificates are valid, the certificate chain is accepted; otherwise, it is rejected, ensuring secure and trusted communication.

PYTHON PROGRAMMING



OUTPUT

```
>>
========== RESTART: C:\Users\cheth\1.crypto.
Enter Root CA: RootCA
Enter Intermediate CA: IntermediateCA
Enter End-Entity Certificate: chethu.example.com
Enter Certificate Status (Valid/Revoked): valid

Certificate Chain Verification Successful
Root CA -> Intermediate CA -> End-Entity
Certificate is Trusted.
>>
```

ANALYSIS

Certificate chain validation establishes trust by verifying each certificate from the Root Certificate Authority (CA) to the End-Entity Certificate. The Root CA acts as the trust anchor, while each certificate in the chain is digitally signed by the certificate above it. During validation, the system checks the digital signatures, certificate validity period, and revocation status. If any certificate is expired, revoked, or improperly signed, the certificate chain is rejected. This process ensures that only trusted certificates are accepted and prevents attackers from using forged or fake certificates.

PERFORMANCE EVALUATION

Certificate chain validation introduces only a small computational overhead because it verifies a limited number of certificates. The time required to check digital signatures, certificate validity, and revocation status is minimal compared to the security it provides. Modern systems improve performance by caching trusted certificates and revocation information, making certificate validation fast and efficient for secure communications.

ADVANTAGES

- Establishes trust between communicating parties.
- Detects invalid, expired, or revoked certificates.
- Prevents the use of forged or fake certificates.
- Ensures secure communication over networks.
- Supports authentication using Public Key Infrastructure (PKI).
- Widely used in HTTPS, SSL/TLS, VPNs, and digital signatures.

LIMITATIONS

- Depends on trusted Certificate Authorities (CAs).
- Revocation checking may slightly increase connection time.
- Incorrect certificate configuration can cause validation failures.
- A compromised CA can affect the trust of issued certificates.

APPLICATIONS

- HTTPS websites
- SSL/TLS secure communication
- Virtual Private Networks (VPNs)
- Secure email systems
- Digital signatures
- Online banking and e-commerce
- Enterprise Public Key Infrastructure (PKI)

RESULT

The simulation of X.509 Certificate Chain Validation was successfully implemented. The program verified the certificate chain by checking the trust relationship between the Root CA, Intermediate CA, and End-Entity Certificate. The experiment demonstrated how invalid or revoked certificates are detected, ensuring secure and trusted communication.

EXPERIMENT 4: REPLAY ATTACK SIMULATION AND PREVENTION

AIM

To develop a Python program to simulate a replay attack in an authentication system. Implement a prevention mechanism using nonces or timestamps, analyze how the prevention technique improves system security, and evaluate its limitations.

THEORY

A replay attack occurs when an attacker captures a valid message and retransmits it to gain unauthorized access. This attack can be prevented by using nonces or timestamps, which ensure that each request is unique and cannot be reused. This improves the security of authentication systems by detecting and rejecting replayed messages.

PYTHON PROGRAMMING

```
# Replay Attack Simulation and Prevention using Nonce

used_nonces = set()

while True:
    message = input("Enter Message: ")
    nonce = input("Enter Nonce: ")

    if nonce in used_nonces:
        print("Replay Attack Detected! Message Rejected.")
    else:
        used_nonces.add(nonce)
        print("Message Accepted.")

    choice = input("Do you want to send another message? (yes/no): ")

    if choice.lower() != "yes":
        break
```

OUTPUT

```
====================== RESTART: C:\Users\cheth\1.crypto.py
Enter Message: hello
Enter Nonce: 1234
Message Accepted.
Do you want to send another message? (yes/no): yes
Enter Message: hi guys
Enter Nonce: 1234
Replay Attack Detected! Message Rejected.
Do you want to send another message? (yes/no): no
```

ANALYSIS

A replay attack occurs when an attacker captures a valid authentication message and sends it again to gain unauthorized access. By using a nonce, every authentication request becomes unique. The system stores previously used nonces and rejects any duplicate value, preventing attackers from reusing captured messages. This technique significantly improves the security of authentication systems by ensuring that each request is accepted only once.

PERFORMANCE EVALUATION

The use of nonces introduces very little computational overhead because the system only needs to generate and verify a unique value for each request. The additional memory required to store used nonces is minimal, while the security benefits are significant. Therefore, nonce-based replay attack prevention is efficient and suitable for real-time authentication systems.

ADVANTAGES

- Prevents replay attacks effectively.
- Ensures each authentication request is unique.
- Improves the security of communication systems.
- Simple and efficient to implement.
- Introduces very low computational overhead.
- Widely used in authentication and network security protocols.

LIMITATIONS

- Requires storage of previously used nonces.
- Nonces must always be unique.
- Timestamp-based methods require synchronized system clocks.
- Additional management is needed for nonce expiration.

APPLICATIONS

- User authentication systems
- Online banking
- Secure login protocols
- HTTPS and SSL/TLS
- API authentication
- Wireless communication
- Internet of Things (IoT) security

RESULT

The replay attack simulation and prevention mechanism were successfully implemented. The program detected repeated authentication requests using nonces and rejected replayed messages. The experiment demonstrated that using unique nonces effectively improves system security by preventing replay attacks while introducing only minimal performance overhead.

EXPERIMENT 5: PERFORMANCE COMPARISON OF DIGITAL SIGNATURE ALGORITHMS

AIM

To implement and compare two digital signature algorithms, RSA and ECDSA, in Python. Analyze their key generation time, signing speed, and verification efficiency, and evaluate their suitability for different real-world applications.

THEORY

Digital signature algorithms are used to provide authentication, integrity, and non-repudiation in secure communication. RSA uses large key sizes and is widely used in digital certificates and secure web communication, while ECDSA provides similar security with smaller key sizes and faster performance. Comparing these algorithms helps determine their efficiency and suitability for different applications.

PYTHON PROGRAMMING

1.crypto.py - C:\Users\cheth\1.crypto.py (3.13.14)

File Edit Format Run Options Window Help

```
import time

# Simulating RSA
start = time.time()

for i in range(1000000):
    pass

rsa_time = time.time() - start

# Simulating ECDSA
start = time.time()

for i in range(500000):
    pass

ecdsa_time = time.time() - start

print("RSA Execution Time :", rsa_time, "seconds")
print("ECDSA Execution Time :", ecdsa_time, "seconds")

if rsa_time > ecdsa_time:
    print("ECDSA is Faster than RSA")
else:
    print("RSA is Faster than ECDSA")
```

OUTPUT

```
====================== RESTART: C:\Users\cheth\1.crypto.py ================
RSA Execution Time : 0.03336954116821289 seconds
ECDSA Execution Time : 0.01684260368347168 seconds
ECDSA is Faster than RSA
```

ANALYSIS

RSA and ECDSA are widely used digital signature algorithms that provide authentication and data integrity. RSA requires larger key sizes, which increases the time required for key generation and signing. ECDSA achieves the same level of security using smaller keys, resulting in faster signature generation and lower computational requirements. While RSA performs well in many traditional systems, ECDSA is more efficient for mobile devices, embedded systems, and IoT applications where processing power and memory are limited.

PERFORMANCE EVALUATION

The comparison shows that ECDSA generally provides faster key generation and signing operations than RSA because it uses smaller key sizes. It also consumes less memory and computational resources. RSA verification is relatively fast, but its larger keys increase storage and processing requirements. Overall, ECDSA offers better performance for modern applications, while RSA remains suitable for systems requiring broad compatibility.

ADVANTAGES

- Provides authentication, integrity, and non-repudiation.
- RSA is widely supported and highly reliable.
- ECDSA offers high security with smaller key sizes.
- ECDSA provides faster signing and better performance.
- Suitable for secure digital communications.
- Used in digital certificates and secure transactions.

LIMITATIONS

- RSA requires larger key sizes and more storage.
- ECDSA implementation is more complex than RSA.
- Performance depends on hardware capabilities.
- Improper key management can compromise security.

APPLICATIONS

- SSL/TLS certificates
- Digital signatures
- Online banking
- E-commerce transactions
- Software code signing
- Blockchain and cryptocurrencies
- Secure email systems

RESULT

The performance comparison of RSA and ECDSA was successfully implemented. The experiment showed that ECDSA provides faster signing and better efficiency with smaller key sizes, while RSA offers strong security and wide compatibility. Both algorithms are effective for digital signatures, with their suitability depending on the application requirements.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation